CFA INSTA
Institut National Supérieur des Technologies Avancées

python

Aller un peu plus loin

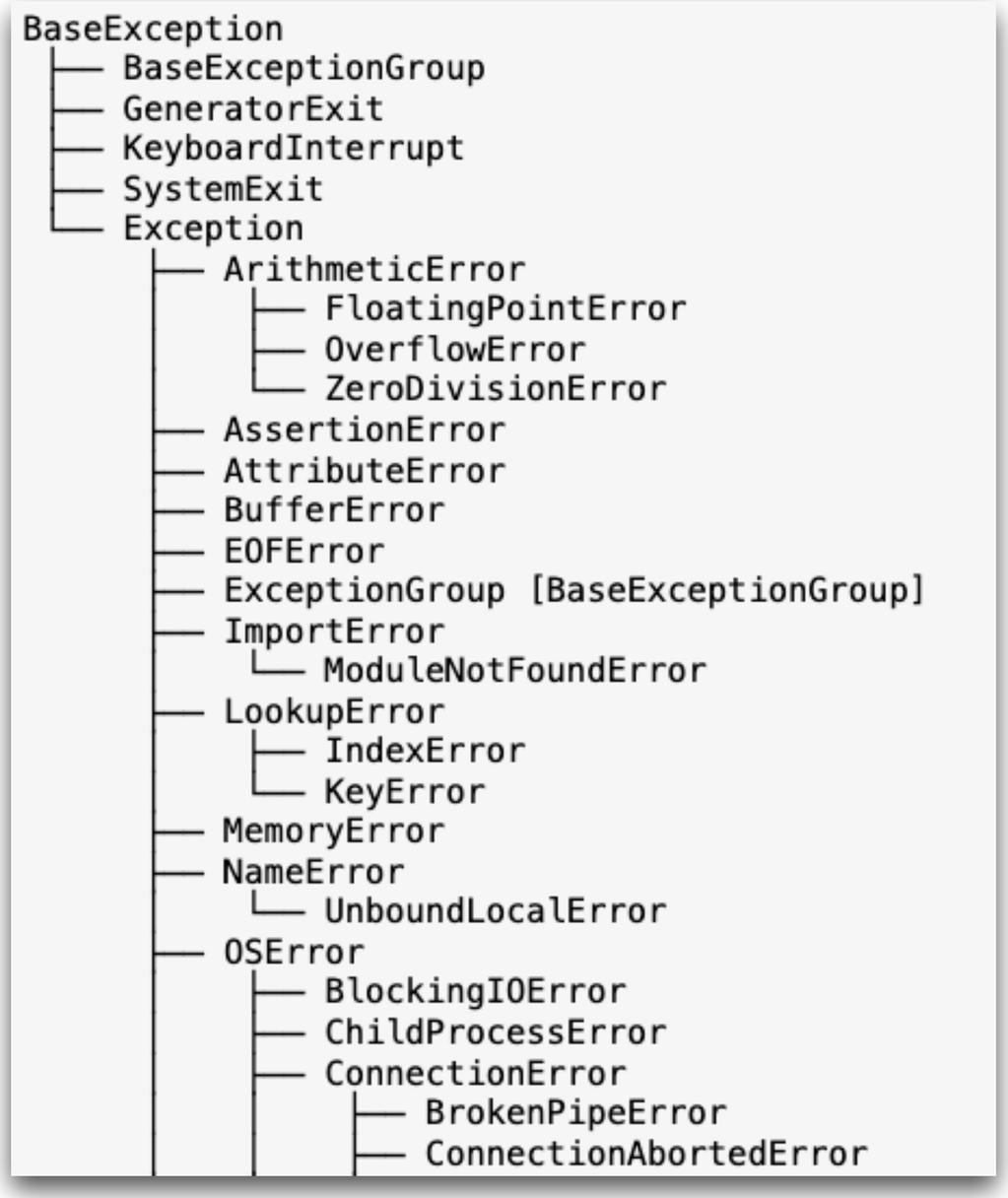


2024

Python

Exceptions

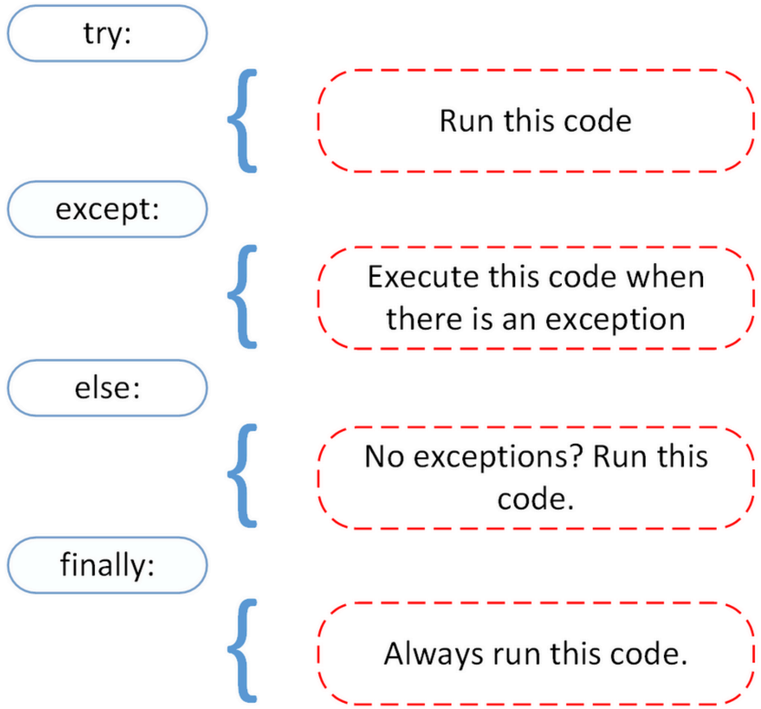
- Une exception est une condition qui survient lors de l'exécution et qui indique qu'un événement inattendu est arrivé
- **Try Except** est un moyen de gérer les exception/erreurs qui peuvent arriver dans le programme
- En python, toutes les exception sont dérivées de la classe **Exception**.



➡https://docs.python.org/3/library/exceptions.html#exception-hierarchy

Python

capture d'exception : structure




```
>>> try:
...     print(2/0)
... except ZeroDivisionError:
...     print("Impossible voyons!")
...
Impossible voyons!
```

- Dans la partie TRY, on mettra le code dont on suspecte qu'il puisse générer une exception

```
>>> mylist = [1, 2, 3, 4, 28]
>>> mylist[8]
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    mylist[8]
    ~~~~~~^^^
IndexError: list index out of range
>>> try:
...     print(mylist[8])
... except IndexError:
...     print("doesn't exists bro!")
...
doesn't exists bro!
```

Python

capture d'exception : types d'erreurs

Nom de l'exception	Description	exemple
SyntaxError	Erreur dans la syntaxe du langage, l'interpréteur va se stopper	pritn('test')
KeyError	Clé non trouvée	d = { 'a': 1} d['b']
IndexError	problème d'index ou index hors de portée	lst = [1, 2, 3] lst[10]
KeyboardInterrupt	Arrive quand l'utilisateur stoppe l'exécution avec un Ctrl+C	Pressing control+c

```
>>> from time import sleep
... while True:
...     try:
...         print("Try and stop me")
...         sleep(1)
...     except:
...         print("Sorry, I'm good")
```

Python

capture d'exception : exercice

- Modifier ce code pour prendre en compte l'exception renvoyée et ajouter un message approprié pour l'utilisateur

```
import json
user_json = '{"name": "Santiago", "age": 35}'
user = json.loads(user_json)

print(user['name'])
print(user['age'])
print(user['address'])
```

Python

capture d’exception : correction

```
import json
user_json = '{"name": "Santiago", "age": 35}'
user = json.loads(user_json)
try:
    print(user['name'])
    print(user['age'])
    print(user['address'])
except KeyError as e:
    print("There are missing fields in the user object: ", e)
    # Properly handle the error
```

Python

Créer vos propres exceptions

- Essayer d'abord ça :

```
class weirdError(Exception):
    print("ok it'es a weird error")
    pass

def strange():
    number = input("enter your favorite number : ")

    if int(number) == 42:
        raise weirdError("weird error spotted")

try:
    strange()
except(weirdError):
    print("it's a weird error")
else:
    print("all good")
```

- puis ajouter ça

```
try:
    strange()
except Exception as e:
    print(e)
else:
    print("all good")
```

```
try:
    strange()
except Exception:
    traceback.print_exc()
else:
    print("all good")
```

Envie d'en savoir plus ?
➡https://docs.python.org/3/library/exceptions.html

Python

fonctions : arguments forcés

- Lorsqu'on appelle une fonction, on peut forcer l'utilisation des arguments
  - parfois certaines fonctions peuvent prendre beaucoup d'arguments

```
>>> def myfunc(*, a, b):
...     print(a, b)
...
>>> myfunc(2, 3)
Traceback (most recent call last):
  File "<python-input-7>", line 1, in <module>
    myfunc(2, 3)
    ~~~~~~^^^^^^
TypeError: myfunc() takes 0 positional arguments but 2 were given
>>> myfunc(a=2,b= 3)
2 3
>>>
>>> args={"a":2,"b":3}
>>> myfunc(**args)
2 3
```

```
>>> def func2(a, b, c):
...     print(a, b, c)
...
>>> list=[2, 4, 3]
>>> func2(*list)
2 4 3
```

- En utilisant * et **, on peut passer en argument un dictionnaire (**) ou une liste (*)

Python

fonctions : décorateurs

- Les décorateurs sont des fonctions qui modifient le comportement d'autres fonctions
- Ils sont utiles pour ajouter un même code à plusieurs fonctions

```python
def print_argument(func):
    def wrapper(choosen_number):
        print("arguments for ", func.__name__, "are", choosen_number)
        return func(choosen_number)
    return wrapper

@print_argument
def multi(a):
    return(a+a)

print(multi(3))
```

Python

Lambda

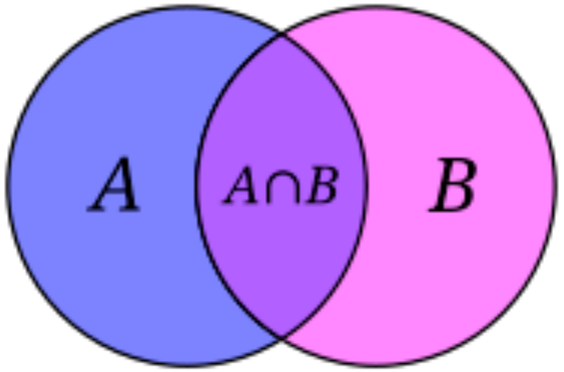
- Parfois, on a pas le souhait de nommer une fonction, notamment si la fonction sera peu utilisée, on peut donc utiliser des fonctions anonymes : les lambdas

```
>>> carre = lambda x : x*x
>>> carre(3)
9
```

```
>>> list=["Amy", "Raymond", "Gina", "perralta", "pimento", "boyle"]
>>> sorted(list, key=lambda x:len(x))
['Amy', 'Gina', 'boyle', 'Raymond', 'pimento', 'perralta']
```

Python

Compréhensions de listes



- La théorie des ensembles est la branche de la logique mathématique qui étudie les ensembles , qui peuvent être décrits de manière informelle comme des collections d'objets *(source : wikipedia)*
- la définition d'ensembles par propriétés est également connue sous le nom de compréhension d'ensemble
- Exemple : $\{n \mid \exists k \in Z, n = 2k\}$
- Inspiré par ce principe, Python propose la notion de compréhension de liste afin de proposer des alternatives aux boucles for

Python

Compréhensions de listes : exemples

```
>>> list=[]
>>> for x in range(0,10):
...     list.append(x)
...
>>> list
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [x for x in range(0,10)]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [x for x in range(0,10) if x%2==0]
[0, 2, 4, 6, 8]
```

- ici, **if** agit comme un filtre

```
>>> list(range(0,10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> ["coucou "+x for x in ["Amy", "Jake", "Gina"]]
['coucou Amy', 'coucou Jake', 'coucou Gina']
```

Python

Documentation

```python
class myclass():
    def myfunc():
        """ this is an amazing function """
        #this is a comment
        print("doing something")


help(myclass)
```

```
Help on class myclass in module __main__:

class myclass(builtins.object)
 |  Methods defined here:
 |
 |  myfunc()
 |      this is an amazing function
 |
 |  ----------------------------------------
```

Python

ouvrir un fichier

```
f=open('monologue.txt')
print(f.read())
f.close()
```

- open() est une fonction built-in, c'est à dire qu'il n'y a pas besoin d'importer quoi que ce soit pour l'utiliser
- ouvrir un fichier utilise des ressources qui ne sont pas libérées si on ne le referme pas.

```
→  python_tp py readtext.py
Mais, vous savez, moi je ne crois pas qu'il y ait de bonne ou de mauvaise situation. Moi, si je devais résumer ma vie auj
ourd'hui avec vous, je dirais que c'est d'abord des rencontres, des gens qui m'ont tendu la main, peut-être à un moment o
ù je ne pouvais pas, où j'étais seul chez moi. Et c'est assez curieux de se dire que les hasards, les rencontres forgent
une destinée… Parce que quand on a le goût de la chose, quand on a le goût de la chose bien faite, le beau geste, parfois
 on ne trouve pas l'interlocuteur en face, je dirais, le miroir qui vous aide à avancer. Alors ce n'est pas mon cas, comm
e je le disais là, puisque moi au contraire, j'ai pu ; et je dis merci à la vie, je lui dis merci, je chante la vie, je d
anse la vie… Je ne suis qu'amour ! Et finalement, quand beaucoup de gens aujourd'hui me disent : « Mais comment fais-tu p
our avoir cette humanité ? » Eh bien je leur réponds très simplement, je leur dis que c'est ce goût de l'amour, ce goût d
onc qui m'a poussé aujourd'hui à entreprendre une construction mécanique, mais demain, qui sait, peut-être simplement à m
e mettre au service de la communauté, à faire le don, le don de soi…
```

Python

ouvrir un fichier avec *with*

```
with open('monologue.txt') as f:
    text = f.read()

print(text)
```

- de cette manière, la ressource ne sera dispo que dans le scope de *with* et la fermeture sera automatique
- utiliser *with* est une meilleure pratique

Caractère	Signification
r	Ouvrir le fichier en lecture (par défaut).
w	Ouvrir le fichier en écriture, en le vidant d'abord.
x	Créer un nouveau fichier et l'ouvrir en écriture.
a	Ouvrir pour écriture, ajouter à la fin du fichier s'il existe déjà.

Python

Écrire

```
with open('my_cool_story.txt', 'w') as f:
    for i in range(1, 5):
        f.write(str(i))

with open('my_cool_story.txt', 'r') as f:
    print(f.read())
```

```
→  python_tp py readtext.py
1234
```

```
with open('my_cool_story.txt', 'r') as f:
    for line in f:
        print(line)
```

Python

opérations sur les fichiers

```
import os
if os.path.isfile('my_cool_story.txt'):
    print("It's a file")
```

```
import os
if os.path.isdir('my_cool_dir'):
    print("It's a directory")
```

commande	description
os.mkdir()	créer un dossier
os.remove()	supprimer un fichier
os.rename('old file.txt','new name.txt')	renommer un fichier
shutil.move('path1','path 2')	déplacer un fichier
shutil.copy()	copier
shutil.copytree()	copier l'arborescence
shutil.rmtree()	supprimer l'arborescence

Python
Threads

- Un processus est une instance d'un programme informatique en cours (programme + données associées + contexte d'exécution)
- un thread est une entité au sein d'un processus dont l'exécution peut être planifiée
- Executer le code suivant
  Que pouvons-nous constater ?



```python
from time import sleep

def killSarahConnor():
    sleep(2)
    print("Sarah Connor terminated")

def fart():
    sleep(1)
    print("Done")

def countToTen():
    for x in range(10):
        print(x)
        sleep(0.5)

countToTen()
killSarahConnor()
fart()
```

Python

Threads

- Voici un exemple avec des trends :

- Ajouter un ***print*** pour signifier que tous les traitements sont terminés

```python
import threading
from time import sleep

def killSarahConnor():
    sleep(2)
    print("Sarah Connor terminated")

def fart():
    sleep(1)
    print("Done")

def countToTen():
    for x in range(11):
        print(x)
        sleep(0.5)

t1 = threading.Thread(target=killSarahConnor)
t1.start()

t2 = threading.Thread(target=fart)
t2.start()

t3 = threading.Thread(target=countToTen)
t3.start()
```

Python

Récursivité

- La récursivité est une technique de programmation ou une fonction s'appelle elle-même

```python
def facto(n):
    # Cas de base
    if n == 0:
        return 1
    # Appel récursif
    else:
        return n * facto(n - 1)

print(facto(5))
```

Python

Connecteurs (sockets)